Ход выполнения лабораторной работы №16

Инкапсуляция и фильтрация

Инкапсуляция полей

Массив книг был заменён на два приватных списка List<Book> — один для свободных книг, другой для взятых. Поля стали private — внешний код не имеет к ним прямого доступа. Был создан конструктор который принимает массив книг и распределяет их по спискам:

```
class Library
{
    private List<Book> _freeBooks;
    private List<Book> _takenBooks;

    public Library(Book[] books)
    {
        _freeBooks  = new List<Book>();
        _takenBooks = new List<Book>();
        foreach (Book book in books)
        {
            if (book.IsTaken) _takenBooks.Add(book);
            else _freeBooks.Add(book);
        }
    }
}
```

Внешний код не изменился — вызовы library.Borrow(), library.Return(), library.PrintAll() остались прежними. Это демонстрирует главное преимущество инкапсуляции — внутреннее представление можно менять не трогая внешний код.

Класс LibraryFilter

Был создан класс LibraryFilter с фильтрами: статус книги (enum BookStatus), содержание строки в названии, отсутствие строки в названии, автор, начало названия. Каждый фильтр реализован через свойство с проверкой через Debug.Assert — строковые фильтры не могут быть пустой строкой, только null или непустая строка.

Метод PrintFiltered

В класс Library добавлен метод PrintFiltered принимающий LibraryFilter. Метод перебирает все книги и применяет каждый фильтр — если книга не проходит хотя бы один фильтр, она пропускается через continue.

Класс InputLibraryFilter

Класс InputLibraryFilter преобразует пользовательский ввод в LibraryFilter. Пустая строка преобразуется в null. Для статуса есть метод TrySetStatus возвращающий false если введено неверное значение.

Результат

Все фильтры отработали корректно:

- Фильтр свободных книг показал все 5 книг после возврата "1984"
- Фильтр по содержанию "принц" нашёл "Маленький принц"
- Фильтр по автору "Оруэлл" нашёл "1984"
- InputLibraryFilter с фильтром по началу "М" и статусу "free" нашёл "Мастер и Маргарита" и "Маленький принц"

```
valentinaadvahova@Valentinas-MacBook-Air MySecondProject % dotnet run
=== Все книги ===
  [свободна] Мастер и Маргарита (1967) — Булгаков
  [свободна] 1984 (1949) — Оруэлл
  [свободна] Гарри Поттер (1997) — Роулинг
  [свободна] Преступление и наказание (1866) — Достоевский
  [свободна] Маленький принц (1943) — Экзюпери

=== Берём '1984' ===
Книга взята: 1984
  [свободна] Мастер и Маргарита (1967) — Булгаков
  [свободна] Гарри Поттер (1997) — Роулинг
  [свободна] Преступление и наказание (1866) — Достоевский
  [свободна] Маленький принц (1943) — Экзюпери
  [занята]   1984 (1949) — Оруэлл

=== Возвращаем '1984' ===
Книга возвращена: 1984

=== Фильтр: свободные книги ===
  [свободна] Мастер и Маргарита (1967) — Булгаков
  [свободна] Гарри Поттер (1997) — Роулинг
  [свободна] Преступление и наказание (1866) — Достоевский
  [свободна] Маленький принц (1943) — Экзюпери
  [свободна] 1984 (1949) — Оруэлл

=== Фильтр: имя содержит 'принц' ===
  [свободна] Маленький принц (1943) — Экзюпери

=== Фильтр: автор Оруэлл ===
  [свободна] 1984 (1949) — Оруэлл

=== InputLibraryFilter тест ===
  [свободна] Мастер и Маргарита (1967) — Булгаков
  [свободна] Маленький принц (1943) — Экзюпери
valentinaadvahova@Valentinas-MacBook-Air MySecondProject %
```

Вывод

В ходе лабораторной работы была реализована инкапсуляция — внутреннее хранение книг изменилось с одного массива на два списка, но внешний код не потребовал изменений. Была создана система фильтрации с контрактами через свойства и Debug.Assert.

Код:

Library.cs

```
using System.Diagnostics;

enum BookStatus { Any, Taken, Free }

class LibraryFilter

{

    private BookStatus _status = BookStatus.Any;

    private string? _nameContains = null;

    private string? _nameNotContains = null;

    private string? _author = null;
```

```
private string? _nameStartsWith = null;

public BookStatus Status
{
    get => _status;
    set
    {
        Debug.Assert(Enum.IsDefined(typeof(BookStatus), value));
        _status = value;
    }
}

public string? NameContains
{
    get => _nameContains;
    set
    {
        Debug.Assert(value == null || value.Length > 0);
        _nameContains = value;
    }
}

public string? NameNotContains
```

```
{
    get => _nameNotContains;
    set
    {
        Debug.Assert(value == null || value.Length > 0);
        _nameNotContains = value;
    }
}

public string? Author
{
    get => _author;
    set
    {
        Debug.Assert(value == null || value.Length > 0);
        _author = value;
    }
}

public string? NameStartsWith
{
    get => _nameStartsWith;
    set
```

```
        {
            Debug.Assert(value == null || value.Length > 0);
            _nameStartsWith = value;
        }
    }
}

class InputLibraryFilter
{
    public LibraryFilter Filter { get; } = new();

    public void SetNameContains(string input)
    {
        Filter.NameContains = input.Length == 0 ? null : input;
    }

    public void SetNameNotContains(string input)
    {
        Filter.NameNotContains = input.Length == 0 ? null : input;
    }

    public void SetAuthor(string input)
    {
```

```
        Filter.Author = input.Length == 0 ? null : input;
    }

    public void SetNameStartsWith(string input)
    {
        Filter.NameStartsWith = input.Length == 0 ? null : input;
    }

    public bool TrySetStatus(string input)
    {
        if (input == "any")  { Filter.Status = BookStatus.Any;   return true; }
        if (input == "taken") { Filter.Status = BookStatus.Taken; return true; }
        if (input == "free")  { Filter.Status = BookStatus.Free;  return true; }
        return false;
    }
}

class Library
{
    private List<Book> _freeBooks;
    private List<Book> _takenBooks;

    public Library(Book[] books)
```

```
    {
        _freeBooks  = new List<Book>();
        _takenBooks = new List<Book>();
        foreach (Book book in books)
        {
            if (book.IsTaken)
                _takenBooks.Add(book);
            else
                _freeBooks.Add(book);
        }
    }

    public void PrintAll()
    {
        foreach (Book book in _freeBooks)
            Console.WriteLine($"  [свободна] {book.Title} ({book.Year}) — {book.Author}");
        foreach (Book book in _takenBooks)
            Console.WriteLine($"  [занята]   {book.Title} ({book.Year}) — {book.Author}");
    }

    public void Borrow(string title)
    {
```

```
    for (int i = 0; i < _freeBooks.Count; i++)
    {
        if (_freeBooks[i].Title == title)
        {
            Book book = _freeBooks[i];
            book.IsTaken = true;
            _freeBooks.RemoveAt(i);
            _takenBooks.Add(book);
            Console.WriteLine("Книга взята: " + title);
            return;
        }
    }
    foreach (Book book in _takenBooks)
    {
        if (book.Title == title)
        {
            Console.WriteLine("Книга уже занята: " + title);
            return;
        }
    }
    Console.WriteLine("Книга не найдена: " + title);
}
```

```
public void Return(string title)
{
    for (int i = 0; i < _takenBooks.Count; i++)
    {
        if (_takenBooks[i].Title == title)
        {
            Book book = _takenBooks[i];
            book.IsTaken = false;
            _takenBooks.RemoveAt(i);
            _freeBooks.Add(book);
            Console.WriteLine("Книга возвращена: " + title);
            return;
        }
    }
    foreach (Book book in _freeBooks)
    {
        if (book.Title == title)
        {
            Console.WriteLine("Книга и так свободна: " + title);
            return;
        }
    }
    throw new ArgumentException("Такой книги не существует: " + title);
```

```
    }

    public void PrintFiltered(LibraryFilter filter)
    {
        List<Book> all = new();
        all.AddRange(_freeBooks);
        all.AddRange(_takenBooks);

        foreach (Book book in all)
        {
            if (filter.Status == BookStatus.Taken && !book.IsTaken) continue;
            if (filter.Status == BookStatus.Free  &&  book.IsTaken) continue;
            if (filter.NameContains    != null && !
book.Title.Contains(filter.NameContains))       continue;
            if (filter.NameNotContains != null &&
book.Title.Contains(filter.NameNotContains))    continue;
            if (filter.Author          != null &&  book.Author != filter.Author)
continue;
            if (filter.NameStartsWith  != null && !
book.Title.StartsWith(filter.NameStartsWith))   continue;

            string status = book.IsTaken ? "занята" : "свободна";
            Console.WriteLine($"  [{status}] {book.Title} ({book.Year}) —
{book.Author}");
        }
```

```
    }
}
```

Program.cs

```
Library library = new(new Book[]
{
    new() { Title = "Мастер и Маргарита",       Year = 1967, IsTaken = false, Author = "Булгаков" },
    new() { Title = "1984",                     Year = 1949, IsTaken = false, Author = "Оруэлл" },
    new() { Title = "Гарри Поттер",             Year = 1997, IsTaken = false, Author = "Роулинг" },
    new() { Title = "Преступление и наказание",  Year = 1866, IsTaken = false, Author = "Достоевский" },
    new() { Title = "Маленький принц",          Year = 1943, IsTaken = false, Author = "Экзюпери" },
});

Console.WriteLine("=== Все книги ===");
library.PrintAll();

Console.WriteLine("\n=== Берём '1984' ===");
library.Borrow("1984");
library.PrintAll();
```

```
Console.WriteLine("\n=== Возвращаем '1984' ===");
library.Return("1984");

Console.WriteLine("\n=== Фильтр: свободные книги ===");
LibraryFilter filter1 = new() { Status = BookStatus.Free };
library.PrintFiltered(filter1);

Console.WriteLine("\n=== Фильтр: имя содержит 'принц' ===");
LibraryFilter filter2 = new() { Status = BookStatus.Any };
filter2.NameContains = "принц";
library.PrintFiltered(filter2);

Console.WriteLine("\n=== Фильтр: автор Оруэлл ===");
LibraryFilter filter3 = new() { Status = BookStatus.Any };
filter3.Author = "Оруэлл";
library.PrintFiltered(filter3);

Console.WriteLine("\n=== InputLibraryFilter тест ===");
InputLibraryFilter input = new();
input.SetNameStartsWith("M");
input.TrySetStatus("free");
library.PrintFiltered(input.Filter);
```

```
class Book
{
    public required string Title  { get; set; }
    public required string Author { get; set; }
    public int Year;
    public bool IsTaken;
}
```